Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 28

Разработка простого API на Node.js и Express.js и подключение к MongoDB для хранения данных веб-приложения

Вариант 2 – лендинг IT-конференции

Выполнила: Клычева Г.
Группа: 310102
Репозиторий: https://github.com/karrinkl/devconf-api.git
Проверил: Прудник А.М.

1 ЦЕЛЬ РАБОТЫ И УСЛОВИЕ ЗАДАНИЯ

Цель работы состоит в получении практических навыков создания бэкенд-сервиса на Node.js, Express.js и MongoDB. Требуется разработать REST API, реализующее операции создания, чтения, обновления и удаления (CRUD) для основной сущности проекта. В дальнейшем это API станет источником данных для клиентского приложения из предыдущих лабораторных работ и основой полнофункционального full-stack приложения.

Вариант 2, лендинг IT-конференции DEVCONF 2026, задаёт сущность «Спикер» (Speaker). Схема Mongoose должна содержать обязательные строковые поля name и topic и необязательное поле photoUrl, а набор CRUD-операций должен быть доступен по адресу /api/speakers.

Технические требования задания:

– проект создан командой npm init -y, установлены express, mongoose, dotenv и nodemon;
– строка подключения к базе вынесена в файл .env и читается библиотекой dotenv;
– схема и модель сущности находятся в отдельном файле и используют валидацию required;
– реализованы пять эндпоинтов: список, одна запись по идентификатору, создание, обновление и удаление;
– асинхронная логика написана на async/await с обработкой ошибок в блоках try...catch;
– API возвращает коды ответа 200, 201, 400, 404 и 500.

Помимо обязательных пакетов подключён cors: клиентское приложение работает на другом порту, и без заголовка Access-Control-Allow-Origin браузер заблокирует его запросы к API. Для запуска на Arch Linux без отдельной установки MongoDB добавлен пакет mongodb-memory-server.

2 СТРУКТУРА ПРОЕКТА И ПОДКЛЮЧЕНИЕ К БАЗЕ

Код разделён на слои по ответственности: маршрут связывает адрес и метод запроса с функцией, контроллер обращается к базе через модель, модель описывает документ. Назначение файлов приведено в таблице 1.

Таблица 1 – Структура проекта devconf-api

Файл	Назначение
server.js	Создание приложения Express, промежуточные обработчики, запуск сервера после подключения к базе
config/db.js	Подключение к MongoDB через mongoose.connect, строка берётся из MONGO_URI
models/Speaker.js	Схема и модель сущности «Спикер»
controllers/speakerController.js	Пять асинхронных функций CRUD-операций
routes/speakerRoutes.js	Маршруты /api/speakers и /api/speakers/:id
middleware/errorHandler.js	Обработка несуществующих адресов и разбор ошибок
data/speakers.js	Восемь демонстрационных спикеров из клиентского приложения
scripts/seed.js	Наполнение базы демонстрационными данными
scripts/local.js	Запуск MongoDB и сервера одной командой без Docker
postman/DEVCONF-API.postman_collection.json	Коллекция запросов для проверки API
.env.example	Образец файла настроек, сам .env исключён из репозитория

Строка подключения не хранится в коде. Файл .env содержит две переменные, порт сервера и адрес базы, и указан в .gitignore, поэтому пароль от облачного кластера не попадает в репозиторий. В репозитории лежит только образец .env.example.

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/devconf
```

Подключение выполняет функция connectDB. Если переменная не задана или база недоступна, процесс завершается с понятным сообщением: сервер без базы всё равно не смог бы обработать ни одного запроса.

```
async function connectDB() {
  const uri = process.env.MONGO_URI;
  if (!uri) {
    console.error('Переменная MONGO_URI не задана.');
    process.exit(1);
  }
  try {
    const connection = await mongoose.connect(uri);
    console.log(`MongoDB подключена: ${connection.connection.host}`);
  } catch (error) {
    console.error(`Не удалось подключиться: ${error.message}`);
    process.exit(1);
  }
}
```

В server.js сервер начинает слушать порт только после того, как connectDB завершилась успешно. Промежуточный обработчик express.json разбирает тело запроса в формате JSON, без него в контроллерах POST и PUT объект req.body был бы пустым.

```
app.use(express.json());
app.use(cors());
app.use('/api/speakers', speakerRoutes);
app.use(notFound);
app.use(errorHandler);

async function start() {
  await connectDB();
  app.listen(PORT, () => console.log(`Сервер запущен:
http://localhost:${PORT}`));
}
```

3 СХЕМА MONGOOSE

Схема сущности описана в файле models/Speaker.js. Поля, их типы и правила проверки приведены в таблице 2.

Таблица 2 – Поля схемы Speaker

Поле	Тип	Правила
_id	ObjectId	Создаётся MongoDB автоматически
name	String	Обязательно, от 2 до 120 символов, пробелы по краям обрезаются
topic	String	Обязательно, от 5 до 300 символов, пробелы по краям обрезаются
photoUrl	String	Необязательно, по умолчанию пустая строка
createdAt	Date	Заполняется Mongoose при создании документа
updatedAt	Date	Обновляется Mongoose при каждом изменении

Для обязательных полей вместо значения true передан массив из признака и текста ошибки. Mongoose подставляет этот текст в объект ошибки валидации, и клиент получает сообщение на русском языке, а не служебную строку библиотеки.

```
const speakerSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Имя спикера обязательно'],
      trim: true,
      minlength: [2, 'Имя спикера не короче 2 символов'],
      maxlength: [120, 'Имя спикера не длиннее 120 символов'],
    },
    topic: {
      type: String,
      required: [true, 'Тема доклада обязательна'],
      trim: true,
      minlength: [5, 'Тема доклада не короче 5 символов'],
      maxlength: [300, 'Тема доклада не длиннее 300 символов'],
    },
    photoUrl: { type: String, trim: true, default: '' },
  },
  { timestamps: true, versionKey: false }
);

module.exports = mongoose.model('Speaker', speakerSchema);
```

Параметр timestamps добавляет поля createdAt и updatedAt, по полю createdAt сортируется список спикеров. Параметр versionKey: false убирает служебное поле __v из документов и ответов API. Модель Speaker работает с коллекцией speakers базы devconf.

4 МАРШРУТЫ И КОНТРОЛЛЕРЫ

Маршруты объявлены в routes/speakerRoutes.js. Префикс /api/speakers задан при подключении маршрутизатора в server.js, поэтому в файле указаны только оставшиеся части адреса. Методы с одинаковым путём объединены цепочкой route.

```
router.route('/').get(getSpeakers).post(createSpeaker);
router
  .route('/:id')
  .get(getSpeakerById)
  .put(updateSpeaker)
  .delete(deleteSpeaker);
```

Полный перечень эндпоинтов с назначением и кодами ответа приведён в таблице 3.

Таблица 3 – Эндпоинты API

Метод	URL	Назначение	Коды ответа
GET	/api/speakers	Список всех спикеров, новые выше	200, 500
GET	/api/speakers/:id	Один спикер по идентификатору	200, 400, 404, 500
POST	/api/speakers	Создание спикера	201, 400, 500
PUT	/api/speakers/:id	Обновление переданных полей спикера	200, 400, 404, 500
DELETE	/api/speakers/:id	Удаление спикера	200, 400, 404, 500

Примеры тела запроса и ответа для каждого эндпоинта приведены в таблице 4. Идентификаторы и отметки времени в ответах сокращены.

Таблица 4 – Примеры запросов и ответов

Запрос	Тело запроса	Ответ

POST
/api/speakers
	{
	"name": "Екатерина Сидорук",
	"topic": "Бэкенд на Express: что ломается при росте нагрузки",
	"photoUrl": "/images/speaker-09-1x.webp"
	}
		201
		{
		"message": "Спикер создан",
		"data": {
		"_id": "6723a1f4...",
		"name": "Екатерина Сидорук",
		"topic": "Бэкенд на Express...",
		"photoUrl": "/images/...",
		"createdAt": "2026-10-27T...",
		"updatedAt": "2026-10-27T..."
		}
		}

GET /api/speakers	нет	200 { "count": 9, "data": [{ "_id": "...", ... }] }
GET /api/speakers/:id	нет	200 { "data": { "_id": "6723a1f4...", "name": "Екатерина Сидорук", ... } }

PUT
/api/speakers/:id
	{
	"topic": "Бэкенд на Express: журналирование и обработка ошибок"
	}
		200
		{ "message": "Спикер обновлён",
		"data": { "topic": "Бэкенд на Express: журналирование...", ... } }

DELETE /api/speakers/:id	нет	200 { "message": "Спикер удалён", "data": { "_id": "6723a1f4...", ... } }

POST
/api/speakers
	{
	"photoUrl": "/images/speaker-10-1x.webp"
	}
		400
		{ "message": "Данные не прошли проверку",
		"errors": [
		"Имя спикера обязательно",
		"Тема доклада обязательна"] }

Все контроллеры устроены одинаково: асинхронная функция, обращение к модели внутри блока try и передача ошибки в общий обработчик вызовом next(error) в блоке catch. Так ошибка запроса к базе не обрушивает процесс сервера, а превращается в ответ с подходящим кодом. Ниже приведён контроллер обновления, самый сложный из пяти.

```
async function updateSpeaker(req, res, next) {
  try {
    const { name, topic, photoUrl } = req.body;
    const update = {};
    if (name !== undefined) update.name = name;
    if (topic !== undefined) update.topic = topic;
    if (photoUrl !== undefined) update.photoUrl = photoUrl;

    if (Object.keys(update).length === 0) {
      return res.status(400).json({ message: 'Тело запроса пустое' });
    }

    const speaker = await Speaker.findByIdAndUpdate(req.params.id,
update, {
      new: true,
      runValidators: true,
    });

    if (!speaker) {
      return res.status(404).json({ message: `Спикер ${req.params.id} не
найден` });
    }
    res.status(200).json({ message: 'Спикер обновлён', data: speaker });
  } catch (error) {
    next(error);
  }
}
```

В объект обновления попадают только переданные поля, поэтому запрос с одним полем topic меняет тему и не затирает имя. Параметр new: true возвращает документ после изменения, а runValidators: true заставляет Mongoose проверять правила схемы и при обновлении: без него пустое имя записалось бы в базу. Контроллер создания передаёт в Speaker.create только поля схемы, что не позволяет клиенту подменить _id или createdAt.

5 ОБРАБОТКА ОШИБОК

Ошибки, которые контроллеры передают дальше, разбирает функция errorHandler из middleware/errorHandler.js. Express распознаёт обработчик ошибок по четырём параметрам, поэтому он подключается после всех маршрутов. Соответствие ситуаций и кодов ответа приведено в таблице 5.

Таблица 5 – Коды ответа API

Код	Ситуация	Где формируется
200	Успешное чтение, обновление или удаление	Контроллеры
201	Спикер создан	createSpeaker
400	Нарушены правила схемы (ValidationError)	errorHandler, перечень ошибок в поле errors
400	Идентификатор не является ObjectId (CastError)	errorHandler
400	Пустое тело запроса PUT	updateSpeaker
404	Документ с таким идентификатором не найден	Контроллеры по id
404	Запрос к несуществующему адресу	notFound
500	Любая другая ошибка, например потеря связи с базой	errorHandler

Разделение ошибок клиента и сервера важно для потребителя API: на код 400 клиентское приложение показывает пользователю текст из ответа, а на код 500 выводит общее сообщение и может повторить запрос позже. Подробности серверной ошибки пишутся в журнал сервера и наружу не отдаются.

```
function errorHandler(error, req, res, next) {
  if (error.name === 'ValidationError') {
    return res.status(400).json({
      message: 'Данные не прошли проверку',
      errors: Object.values(error.errors).map((item) => item.message),
    });
  }
  if (error.name === 'CastError' && error.path === '_id') {
    return res.status(400).json({
      message: `Значение ${error.value} не является корректным
идентификатором`,
    });
  }
  console.error(error);
  res.status(500).json({ message: 'Внутренняя ошибка сервера' });
}
```

6 ПРОВЕРКА В POSTMAN

Для проверки подготовлена коллекция Postman из семи запросов, она лежит в каталоге postman репозитория. Базовый адрес хранится в переменной коллекции baseUrl. Первый запрос после успешного создания сохраняет идентификатор нового спикера в переменную speakerId скриптом на вкладке Tests, и следующие запросы подставляют его в адрес сами.

```
pm.test('Код ответа 201', function () {
  pm.response.to.have.status(201);
});
const body = pm.response.json();
pm.collectionVariables.set('speakerId', body.data._id);
```

Перед проверкой база заполнена восемью демонстрационными спикерами из data/speakers.js. Создание нового спикера приведено на рисунке 1.

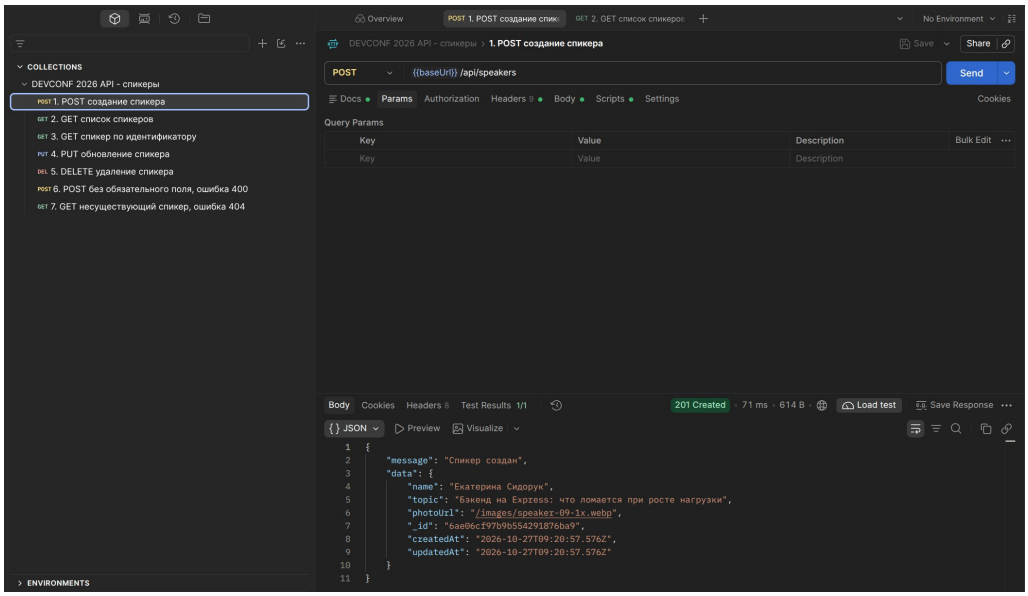


Рисунок 1 – Создание спикера, ответ 201 Created

Сервер вернул код 201 и созданный документ с идентификатором, присвоенным MongoDB, и отметками createdAt и updatedAt. Получение списка спикеров приведено на рисунке 2.

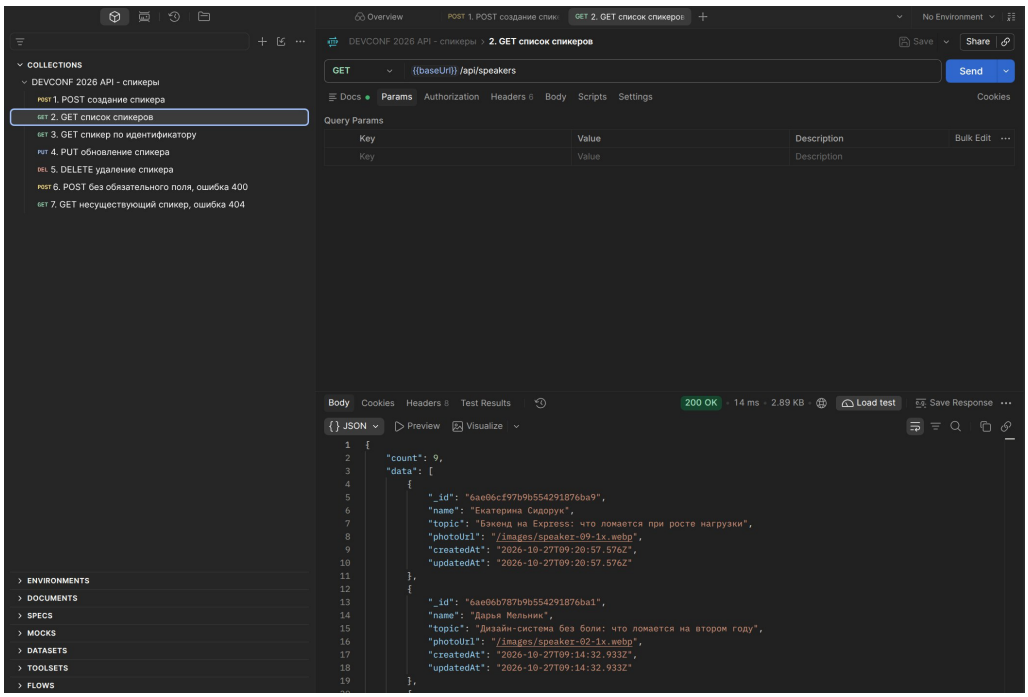


Рисунок 2 – Получение списка спикеров, ответ 200 OK

В ответе поле count равно 9: восемь спикеров из сид-скрипта и один только что созданный, который стоит первым благодаря сортировке по createdAt. Получение одного спикера по идентификатору приведено на рисунке 3.

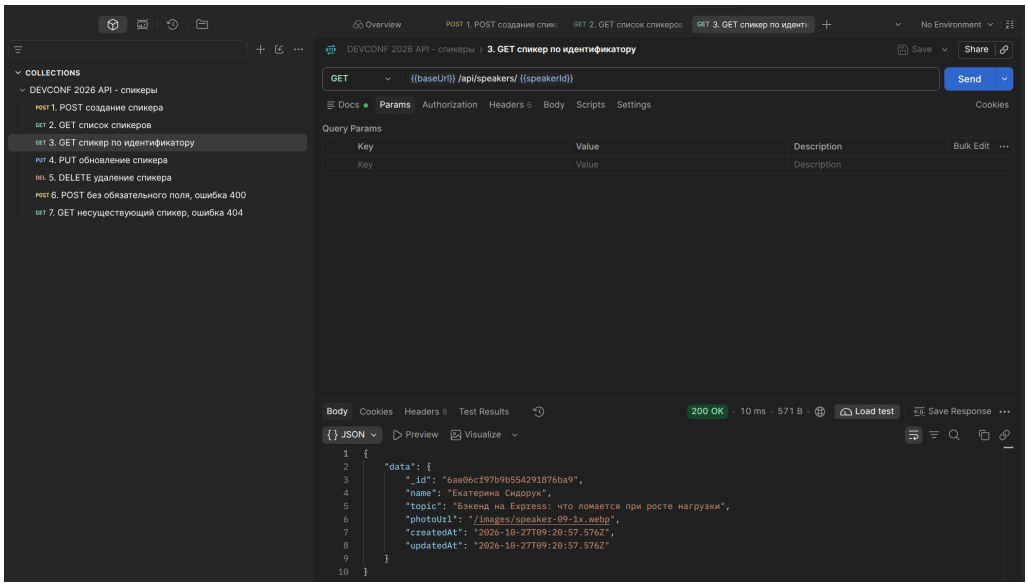


Рисунок 3 – Получение спикера по идентификатору, ответ 200 OK

Идентификатор подставлен из переменной коллекции, сервер вернул тот же документ. Обновление темы доклада приведено на рисунке 4.

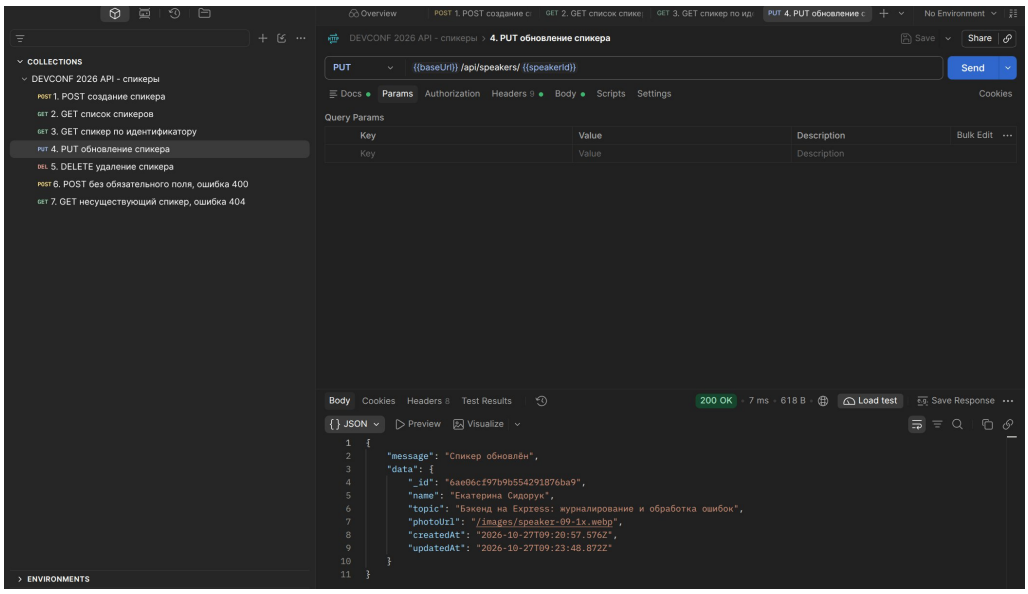


Рисунок 4 – Обновление спикера, ответ 200 OK

В теле запроса передано только поле topic. В ответе тема изменилась, имя осталось прежним, а значение updatedAt стало больше createdAt. Удаление спикера приведено на рисунке 5.

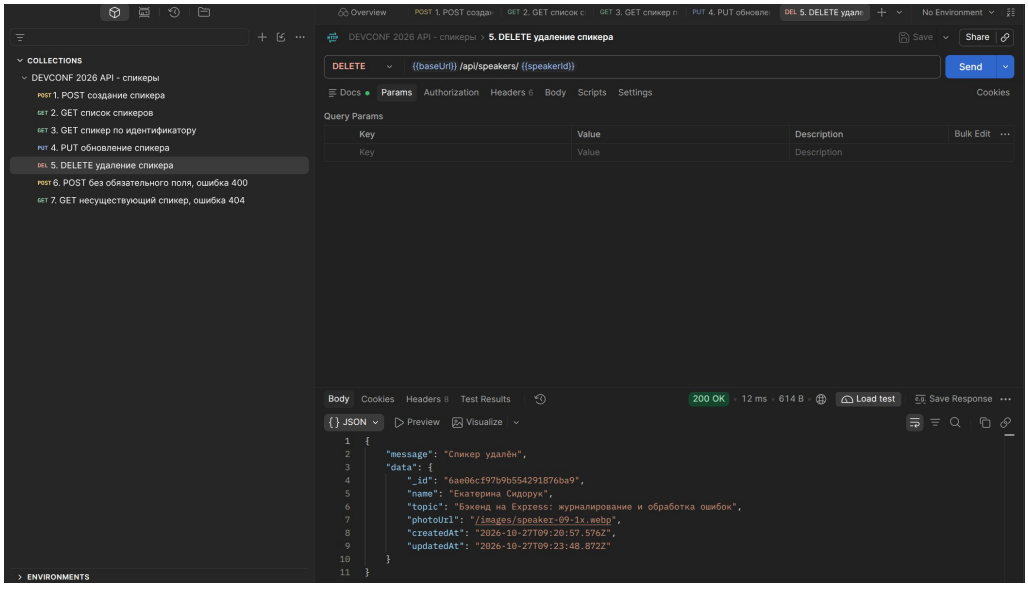


Рисунок 5 – Удаление спикера, ответ 200 OK

Сервер вернул удалённый документ, повторный запрос GET по тому же идентификатору получает ответ 404. Проверка валидации схемы приведена на рисунке 6.

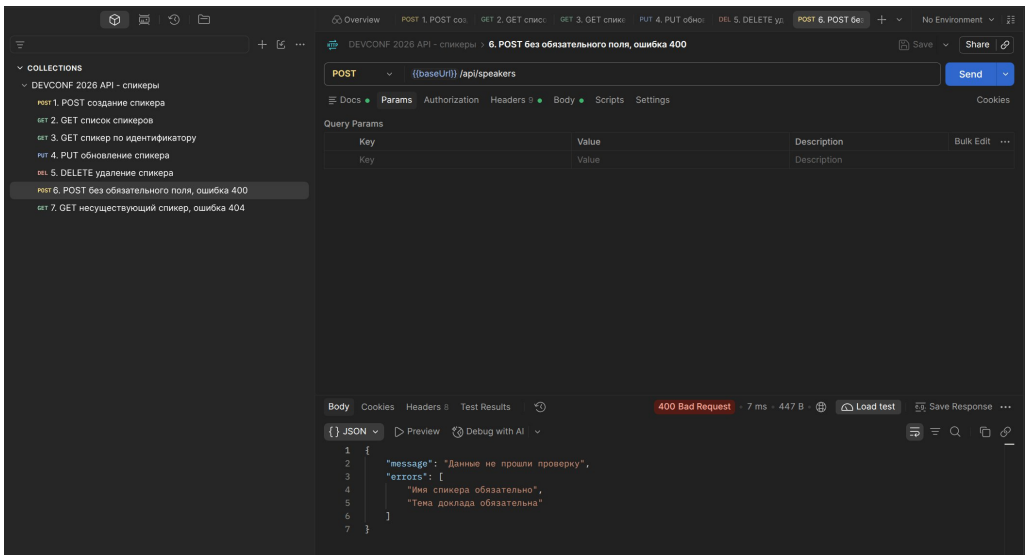


Рисунок 6 – Создание спикера без обязательных полей, ответ 400 Bad Request

Запрос без полей name и topic отклонён с кодом 400, в поле errors перечислены оба нарушенных правила схемы с текстами из Speaker.js. Документ в базу не записан.

Запрос к несуществующему спикеру приведён на рисунке 7.

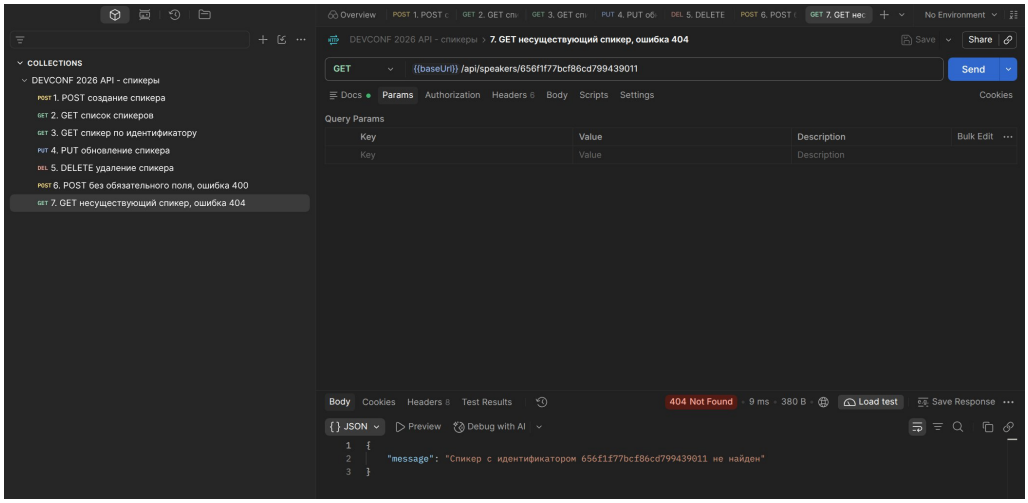


Рисунок 7 – Запрос несуществующего спикера, ответ 404 Not Found

Идентификатор имеет корректный формат ObjectId, но документа с ним нет, поэтому сервер вернул код 404 и сообщение об отсутствии спикера.

7 УСТАНОВКА И ЗАПУСК

Для запуска нужен Node.js версии 18 или новее. Зависимости устанавливаются командами:

```
git clone https://github.com/karrinkl/devconf-api.git
cd devconf-api
npm install
cp .env.example .env
```

Для MongoDB предусмотрено три варианта, они описаны в README.md репозитория. Самый простой не требует ни установки MongoDB, ни Docker: команда npm run local поднимает настоящий сервер mongod через пакет mongodb-memory-server, хранит данные в каталоге .mongo-data, заполняет пустую коллекцию восемью спикерами и запускает API.

```
npm run local
```

Второй вариант использует Docker Compose: файл docker-compose.yml описывает сервисы mongo и api, и сервер ждёт, пока база ответит на проверку готовности. Третий вариант подключается к облачному кластеру MongoDB Atlas заменой строки MONGO_URI в .env. Сводка команд приведена в таблице 6.

Таблица 6 – Команды проекта

Команда	Назначение
npm run local	MongoDB и сервер одной командой, без Docker
npm run dev	Сервер с перезапуском при правках через nodemon
npm start	Обычный запуск сервера
npm run seed	Очистка коллекции и запись восьми спикеров
npm run seed:clear	Только очистка коллекции
docker compose up -d	MongoDB и сервер в контейнерах

После запуска API доступно по адресу http://localhost:5000, корневой адрес возвращает название сервиса и список эндпоинтов.

ЗАКЛЮЧЕНИЕ

Разработан REST-сервис на Node.js и Express.js для сущности «Спикер» лендинга DEVCONF 2026. Данные хранятся в MongoDB, подключение выполняется через Mongoose, строка подключения вынесена в файл .env и читается библиотекой dotenv.

Схема Speaker содержит обязательные поля name и topic с ограничениями длины и необязательное поле photoUrl, сообщения об ошибках валидации заданы на русском языке. Реализованы пять CRUD-эндпоинтов по адресу /api/speakers на async/await с обработкой ошибок в блоках try...catch, общий обработчик возвращает коды 400, 404 и 500 в зависимости от вида ошибки.

Работа API проверена коллекцией запросов Postman: создание, чтение списка и отдельной записи, обновление и удаление выполнены успешно, запрос с нарушением правил схемы отклонён с кодом 400. Сервис запускается одной командой без установки MongoDB и готов служить источником данных для клиентского приложения из лабораторной работы № 25.